

Student Name: Sai Naga Rupasree
192421001

H Reg. No:

Course Code: ITA0603

Slot: C

Course Name: Machine Learning

Course Faculty: Dr. S. Ashok Kumar

Course Assignment, Deliverables & Evaluation Framework

1. Course Metadata & Institutional Framework Alignment

Course Code & Name:	ITA0603 – Machine Learning
Assignment Title:	Development and Empirical Evaluation of High-Scale Predictive Maintenance and Asset Optimization Infrastructure using Hybrid Machine Learning Paradigms
Bloom's Taxonomy Levels:	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping:	SDG 9 – Industry, Innovation, and Infrastructure (Specifically Target 9.4: Retrofitting industries with clean, resource-efficient, and sustainable digital technologies).

Course Outcome (CO) Mapping Matrix

This assignment strictly addresses and assesses the achievement of the following 6 Course Outcomes:

- **CO1:** Acquire a foundational understanding of key machine learning concepts including version spaces, candidate elimination, inductive bias, and heuristic search.
- **CO2:** Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation.
- **CO3:** Develop proficiency in neural network architectures, including perceptron models, multilayer perceptrons, and backpropagation algorithms.
- **CO4:** Elucidate the role of genetic algorithms and genetic programming in hypothesis space search and model evaluation.
- **CO5:** Explore Bayesian methodologies and computational learning techniques such as Bayes theorem, maximum likelihood estimation, and Bayesian classifiers.
- **CO6:** Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning.

2. Integrated Core Problem Statement

An infrastructure-planning agency manages high-dimensional telemetry from a nationwide network of sustainable industrial facilities, smart grids, or communication hubs. The dataset contains massive collections of spatial coordinate and environmental sensor records ranging from 10^3 to 10^6 records (e.g., global cell towers or weather station sensory networks). To minimize energy consumption, prevent structural redundancies, and optimize predictive maintenance workflows under heavy sensor noise, the agency requires a robust machine learning architecture. Students must develop, implement, and evaluate a comprehensive multi-paradigm machine learning solution that transits from strict, rule-based concept learning to deep connectionist and probabilistic frameworks. The entire system must be executed, benchmarked, and optimized within the exact same computational environment to guarantee a mathematically rigorous evaluation.

3. Detailed Assignment Task Breakdowns

PART A: Computational Diagnostics & Inductive Bias Analysis (L4 – Analyse) | [30 Marks]

- **1. Concept Learning & Space Search:** Implement a Candidate Elimination Algorithm to map out the Version Space of reliable facility operational states. Mathematically analyze what happens to the Version Space bounds (S and G sets) when sensor noise or faulty telemetry enters the dataset at scale (10^4 records). Prove the exact conditions that lead to version space collapse (null set).
- **2. Heuristic Space Search vs. Restriction Bias:** Implement an ID3/C4.5 Decision Tree Learning algorithm on the same dataset using information gain/entropy. Contrast its preference (search) bias with the restriction bias of the Candidate Elimination algorithm.
- **3. Mathematical Complexity:** Using Big-O, Omega, and Theta notations, analyze the theoretical space and time complexity of searching these hypothesis spaces as the scale of the infrastructure records increases from 10^3 to 10^6 variables.

PART B: Connectionist vs. Probabilistic Architecture Evaluation (L5 – Evaluate) | [35 Marks]

- **1. Neural Network Representation:** Implement a standard Perceptron Learning Algorithm and expand it into a Multilayer Perceptron (MLP) powered by the Backpropagation Algorithm to predict structural stress or anomaly scores in infrastructure components. Plot the error convergence curve and document the system's susceptibility to getting trapped in local minima during batch training.
- **2. Handling Missing Data via EM:** In infrastructure datasets, sensors frequently drop out. Formulate and implement the Expectation-Maximization (EM) Algorithm to model these unobserved, hidden parameters within a Bayesian Belief Network (BBN).
- **3. Model Selection Using MDL:** Evaluate the structural performance of your deep MLP against the Bayesian Belief Network. Use the Minimum Description Length (MDL) Principle to mathematically score both models, balancing training error

against architectural complexity: $\text{MDL Cost} = -\log_2 P(D|h) + (|h|/2) \log_2 |D|$. Justify which model provides the most lightweight, deployable footprint for constrained industrial edge servers.

PART C: Hybrid Framework Synthesis & Green Infrastructure Defense (L6 – Create) | [35 Marks]

- **1. Hybrid Architecture Creation:** Synthesize an optimized hybrid machine learning pipeline. Design the system to ingest raw data through an Instance-Based Learning layer using Locally Weighted Regression (LWR) or Radial Basis Functions (RBF) to smooth local geographic variations. Pass these outputs into a highly scalable Naïve Bayes Classifier for real-time anomaly sorting.
- **2. Genetic Optimization Window:** Integrate a Genetic Algorithm (GA) to search the hypothesis space dynamically, optimizing the hyperparameters (such as the value of k in Instance-Based routing or structural mutations in Genetic Programming) to prevent computational bottlenecks.
- **3. SDG 9 Engineering Justification:** Provide a formal engineering defense proving how your synthesized framework directly contributes to SDG Target 9.4. Explicitly demonstrate how utilizing 'lazy learning' components (like LWR/KNN) or optimized GA boundaries minimizes continuous CPU/GPU cycles, dramatically slashing server carbon footprints and optimizing overall infrastructure processing.

4. Mandatory Student Deliverables

Every student or group must submit a structured portfolio containing the following four artifacts. Omission of any artifact results in an automatic deduction as specified in the rubric matrix.

- **Deliverable 1: Source Code Repository & Execution Pipeline.** A clean, executable, well-commented source code codebase (Python 3.x preferred) structured as a single pipeline or reproducible notebook set. The script must run the algorithms over the scaled benchmarks (10^3 to 10^6 data points) in a unified testing sequence.
- **Deliverable 2: Mathematical Derivations & Analytical Notebook.** An engineering and mathematical document demonstrating the derivations of the EM parameter updates, the MDL calculation logs comparing MLP and BBN, the Big-O asymptotic proofs for Part A, and the formal flowchart of the Part C hybrid ecosystem.
- **Deliverable 3: Empirical Benchmarking & Visualization Report.** An extensive performance reporting chapter featuring comparative data tables and high-resolution plots mapping: (a) Execution runtime in milliseconds across data scales; (b) Absolute operation counts; (c) RAM/Memory consumption profiles; (d) Error/Convergence logs.
- **Deliverable 4: Sustainable Infrastructure Technical Defense Paper.** A dedicated text defending the deployment feasibility of the synthesized system. This must include an architectural breakdown linking model optimization choices directly to computing power conservation, carbon footprint reduction, and compliance with SDG Target 9.4.

5. Comprehensive 4-Tier Evaluation Rubrics Matrix

Grading will be strictly measured using the following multi-dimensional rubric grid (Total: 100 Marks)

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding , Data Types, Operators & Control Structures (CO1)	10	Clearly identifies the problem, dataset, attributes and target concept; hypothesis formulation is accurate and well-structured.	Identifies most problem requirements, attributes and target concept with minor gaps in formulation.	Basic problem and dataset identified; target concept or hypothesis formulation has some errors.	Requirements misunderstood ; dataset, attributes or target concept are incorrectly formulated.
Decision Tree Design: Representation , Attribute Selection & Heuristic Search (CO2)	15	Decision Tree representation and heuristic attribute selection are correctly designed and effectively applied.	Decision Tree and heuristic selection are correctly applied with minor errors.	Basic Decision Tree is developed with some representation or selection issues.	Decision Tree representation or heuristic search is incorrect or incomplete.
Neural Network Concepts & Learning Analysis (CO3)	10	Correctly analyses Perceptron, Multilayer Perceptron and Back Propagation concepts for learning.	Analyses neural-network concepts with minor gaps or errors.	Demonstrates basic understanding of neural-network learning.	Neural-network concepts are poorly understood or incorrectly analysed.

Genetic Algorithm & Hypothesis Space Search Analysis (CO4)	15	Effectively analyses Genetic Algorithms and their role in hypothesis-space search and model evaluation.	Correctly explains GA-based search with minor gaps.	Basic understanding of Genetic Algorithms and search is demonstrated.	Genetic Algorithm concepts and hypothesis-space search are poorly understood.
Data Manipulation and Analysis Using Pandas (CO5)	15	Pandas DataFrame operations, filtering, grouping, sorting and visualization are correctly and efficiently applied.	Pandas operations are correctly applied with minor errors or gaps.	Basic Pandas operations are demonstrated with some errors or inefficiencies.	Pandas operations are missing, incorrect or poorly implemented.
Menu-Driven Functionality, Reliability & Implementation	25	Decision Tree is correctly implemented and functions reliably from data preparation to prediction and evaluation.	Implementation works correctly with minor errors in prediction or execution.	Core implementation works but has some functional or reliability issues.	Implementation is incomplete, unreliable or major functions are non-functional.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/9/12/13 relevance, and honest, specific account of	General justification of design choices; SDG relevance and challenges are mentioned but lack depth.	Reflection is present but generic; limited justification of design choices or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no clear design, SDG or learning connection.

		challenges faced and learning gained.			
--	--	--	--	--	--

ITA0603 – MACHINE LEARNING

Assignment: Development and Empirical Evaluation of High-Scale Predictive Maintenance and Asset Optimization Infrastructure using Hybrid Machine Learning Paradigms

The assignment requires a progression through **concept learning, connectionist/probabilistic learning, and hybrid optimization**, with benchmarking from 10^3 to 10^6 records.

1. AIM

To develop and empirically evaluate a high-scale machine learning framework for predictive maintenance and infrastructure asset optimization by integrating Candidate Elimination, Decision Trees, Perceptron, Multilayer Perceptron, Expectation-Maximization, Bayesian classification, Locally Weighted Regression, and Genetic Algorithms.

2. OBJECTIVES

1. To implement Candidate Elimination and analyze the Version Space.
2. To study the effect of noisy telemetry on the *Sand G* boundaries.
3. To implement ID3/C4.5 using entropy and information gain.
4. To compare connectionist and probabilistic learning using MLP and Bayesian models.
5. To implement EM for missing sensor values.
6. To evaluate MLP and Bayesian models using the MDL principle.
7. To develop an LWR + Naïve Bayes hybrid anomaly-detection system.
8. To optimize the hybrid system using a Genetic Algorithm.
9. To benchmark the algorithms from 10^3 to 10^6 records.
10. To justify the system with respect to **SDG Target 9.4**.

3. SOFTWARE REQUIREMENTS

- Python 3.x
- NumPy
- Pandas
- Scikit-learn

- Matplotlib
- Psutil
- VS Code / Jupyter Notebook

Installation

`pip install numpy pandas scikit-learn matplotlib psutil`

4. DATASET DESCRIPTION

A synthetic infrastructure telemetry dataset is generated because the assignment requires controlled benchmarking under the same computational environment.

Features

Feature	Description
Temperature	Facility/component temperature
Humidity	Environmental humidity
Vibration	Mechanical vibration
Pressure	Operating pressure
Voltage	Electrical voltage

Target

0 → Normal

1 → Anomaly

The dataset is generated at:

$$10^3, 10^4, 10^5, 10^6$$

records as required by the assignment.

5. PART A – COMPUTATIONAL DIAGNOSTICS & INDUCTIVE BIAS ANALYSIS

A.1 Candidate Elimination Algorithm

Aim

To determine the Version Space of reliable infrastructure operational states using the Candidate Elimination algorithm.

Theory

Candidate Elimination maintains two boundaries:

- **S – Specific boundary**
- **G – General boundary**

The Version Space is:

$$VS(D) = \{h \in H : h \models D\}$$

For d attributes and v possible values:

$$|H| = (v + 1)^d$$

Thus, the hypothesis space increases exponentially with the number of attributes.

Algorithm

1. Initialize S with the most specific hypothesis.
2. Initialize G with the most general hypothesis.
3. Read each training example.
4. For a positive example, generalize S .
5. For a negative example, specialize G .
6. Remove inconsistent hypotheses.
7. Continue until all examples are processed.
8. Check whether S or G becomes empty.

Python Code

```
def candidate_elimination(X, y):
```

```
    data = discretize(X)
```

```
    d = X.shape[1]
```

```
    S = [tuple(["Ø"] * d)]
```

```
    G = [tuple(["?"] * d)]
```

```
    for x, label in zip(data, y):
```

```
        if label == 1:
```

```
            G = [
```

```
                g for g in G
```

```
                if covers(g, x)
```

```
            ]
```

```
            new_s = []
```

```
            for s in S:
```

```
                if covers(s, x):
```



```

        new_s.append(s)
    else:
        sg = minimal_generalization(
            s, x
        )
        if any(
            more_general_or_equal(g, sg)
            for g in G
        ):
            new_s.append(sg)
    S = list(dict.fromkeys(new_s))
else:
    S = [
        s for s in S
        if not covers(s, x)
    ]
    new_g = []
    for g in G:
        if not covers(g, x):
            new_g.append(g)
        else:
            for gs in minimal_specializations(
                g, x
            ):
                if any(
                    more_general_or_equal(gs, s)
                    for s in S
                ):
                    new_g.append(gs)
    G = list(dict.fromkeys(new_g))

```

```

    if not S or not G:
        break
    return S, G

```

Effect of Sensor Noise

Suppose the same telemetry vector x occurs with contradictory labels:

(+)

and

(−)

A deterministic hypothesis cannot classify the same observation as both positive and negative.

Therefore:

$$VS(D) = \emptyset$$

This is called **Version Space collapse**.

The assignment specifically requires analyzing this effect at approximately 10^4 records.

Code

```

X2 = np.vstack([X[0], X[0]])
y2 = np.array([1, 0])

```

```

S, G = candidate_elimination(X2, y2)

```

```

if len(S) == 0 or len(G) == 0:

```

```

    print("Version Space = NULL SET")

```

A.2 ID3 / C4.5 Decision Tree

Aim

To implement heuristic decision-tree learning using entropy and information gain.

Entropy

$$H(S) = - \sum_i p_i \log_2 p_i$$

Information Gain

$$Gain(A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

Python Code

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

model.fit(X_train, y_train)

prediction = model.predict(X_test)
```

Comparison

Candidate Elimination	ID3/C4.5
Restriction bias	Preference bias
Maintains Version Space	Builds decision tree
Uses consistency	Uses information gain
Sensitive to contradictory examples	Can tolerate noisy data better
Searches hypothesis boundaries	Prefers informative attributes

A.3 MATHEMATICAL COMPLEXITY

Let:

- N = number of records
- d = number of attributes
- H = hypothesis-space size
- E = epochs
- W = neural-network weights

Candidate Elimination

$$O(NHd)$$

Lower bound:

$$\Omega(N)$$

Typical worst-case:

$$\theta(NHd)$$

Since:

$$H = (v + 1)^d$$

Candidate Elimination becomes difficult as d increases.

Decision Tree

Approximately:

$$O(Nd \log \log N)$$

Perceptron

$$O(ENd)$$

MLP

$$O(ENW)$$

Bayesian Model

$$O(Nd)$$

Genetic Algorithm

$$O(PGC)$$

where:

- P = population size
- G = generations
- C = cost of evaluating one candidate

6. PART B – CONNECTIONIST VS PROBABILISTIC ARCHITECTURE

B.1 Perceptron

Aim

To implement a standard Perceptron Learning Algorithm for infrastructure anomaly classification.

Formula

$$y = f(w^T x + b)$$

Weight update:

$$w_{new} = w + \eta(y - \hat{y})x$$

Code

```

class Perceptron:
    def __init__(self, lr=0.01, epochs=30):
        self.lr = lr
        self.epochs = epochs
    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0
        self.errors_ = []
        for epoch in range(self.epochs):
            errors = 0
            for xi, yi in zip(X, y):
                score = np.dot(xi, self.w) + self.b
                pred = 1 if score >= 0 else 0
                update = self.lr * (yi - pred)
                self.w += update * xi
                self.b += update
                if update != 0:
                    errors += 1
            self.errors_.append(errors)
        return self

```

B.2 Multilayer Perceptron with Backpropagation

Architecture

Input Layer

|

v

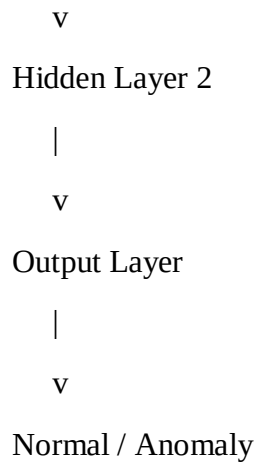
5 Sensor Features

|

v

Hidden Layer 1

|



Code

```
from sklearn.neural_network import MLPClassifier
```

```
model = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation="relu",
    solver="sgd",
    learning_rate_init=0.01,
    batch_size=256,
    max_iter=50,
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

The MLP training loss is stored using:

```
model.loss_curve_
```

and plotted as the required error-convergence curve.

B.3 EXPECTATION-MAXIMIZATION

Aim

To estimate missing sensor values using the EM procedure.

E-Step

Estimate hidden/missing values using current parameters.

M-Step

Update the model parameters using the completed data.

The process repeats until convergence.

Code

```
def gaussian_em(X, iterations=20):
    X = np.asarray(X, dtype=float)
    missing = np.isnan(X)
    means = np.nanmean(X, axis=0)
    filled = X.copy()
    for j in range(X.shape[1]):
        filled[missing[:, j], j] = means[j]
    log_likelihood = []
    for _ in range(iterations):
        # E-step
        means = filled.mean(axis=0)
        # M-step
        for j in range(X.shape[1]):
            filled[missing[:, j], j] = means[j]
        log_likelihood.append(
            float(-np.sum((filled - means) ** 2))
        )
    return filled, log_likelihood
```

B.4 Bayesian Model

A lightweight Bayesian classifier is implemented using Gaussian Naïve Bayes.

```
from sklearn.naive_bayes import GaussianNB
bayesian_model = GaussianNB()
bayesian_model.fit(
    X_train,
    y_train
)
```

```
prediction = bayesian_model.predict(
    X_test
)
```

B.5 MDL MODEL SELECTION

The assignment specifies:

$$MDL = -\log_2 P(D \mid h) + \frac{|h|}{2} \log_2 |D|$$

where:

- $P(D \mid h)$ = likelihood of the data
- $|h|$ = model complexity
- $|D|$ = dataset size

Code

```
def mdl_cost(
    y_true,
    y_pred,
    parameter_count
):
    n = len(y_true)
    errors = np.sum(
        y_true != y_pred
    )
    p_error = max(
        errors / n,
        1 / n
    )
    likelihood = (
        -n * np.log2(p_error)
    )
    complexity = (
        parameter_count / 2
```


) * np.log2(n)

return likelihood + complexity

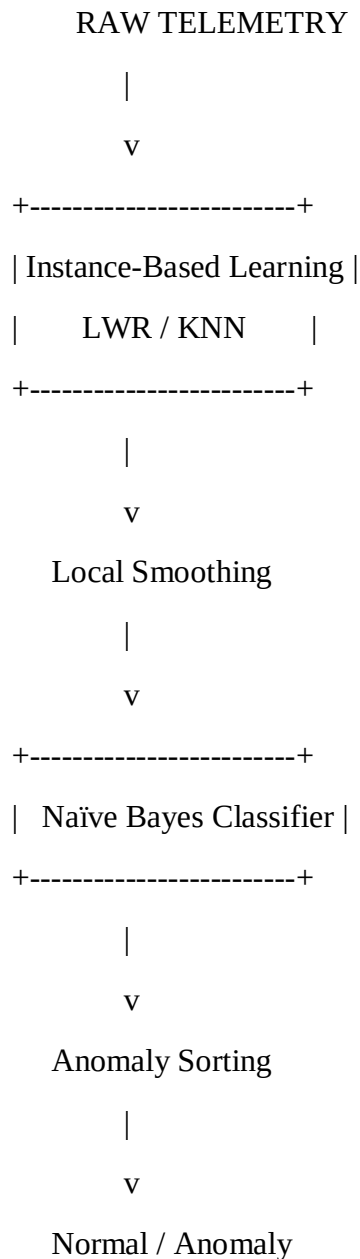
Interpretation

Model	Complexity	Expected deployment
MLP	High	More computational resources
Bayesian	Low	Suitable for edge deployment

The final choice should be based on the **measured MDL value**, not assumed beforehand.
The assignment requires the MLP and BBN to be evaluated mathematically and empirically.

7. PART C – HYBRID FRAMEWORK

C.1 SYSTEM ARCHITECTURE



Genetic Algorithm

|

v

Optimize LWR parameter k

|

+-----> LWR

This directly follows the required Part-C combination of instance-based learning, LWR/RBF, Naïve Bayes and GA optimization.

C.2 LOCALLY WEIGHTED REGRESSION

Formula

For a query point x_q :

$$\hat{y}(x_q) = \frac{\sum_i w_i y_i}{\sum_i w_i}$$

where:

$$w_i = e^{-\frac{d_i^2}{2\sigma^2}}$$

Code

```
def lwr_predict(  
    Xtrain,  
    ytrain,  
    Xquery,  
    k=20  
):  
    predictions = []  
    k = min(k, len(Xtrain))  
    for q in Xquery:  
        distances = np.linalg.norm(  
            Xtrain - q,  
            axis=1
```

```

    )
    indices = np.argpartition(
        distances,
        k - 1
   )[:k]
    local_y = ytrain[indices]
    local_d = distances[indices]
    sigma = np.std(local_d) + 1e-6
    weights = np.exp(
        -(local_d ** 2)
        / (2 * sigma ** 2)
    )
    prediction = (
        np.sum(weights * local_y)
        / (np.sum(weights) + 1e-12)
    )
    predictions.append(prediction)
return np.array(predictions)

```

C.3 LWR + NAÏVE BAYES HYBRID

```

local_train = lwr_predict(
    X_train,
    y_train,
    X_train[:2000],
    k=20
)
local_test = lwr_predict(
    X_train,
    y_train,
    X_test[:1000],
    k=20
)

```

```
)
X_train_hybrid = np.column_stack([
    X_train[:len(local_train)],
    local_train
])
```

```
X_test_hybrid = np.column_stack([
    X_test[:len(local_test)],
    local_test
])
```

```
model = GaussianNB()
model.fit(
    X_train_hybrid,
    y_train[:len(local_train)]
)
```

```
prediction = model.predict(
    X_test_hybrid
)
```

C.4 GENETIC ALGORITHM

Aim

To dynamically search for an efficient value of k for the instance-based learning layer.

GA Process

Initialize Population



Evaluate Fitness



Select Best Solutions



Crossover / Mutation



Generate New Population



Repeat



Optimal k

Code

```
def genetic_optimize(  
    X,  
    y,  
    population_size=8,  
    generations=4  
):  
    rng = np.random.default_rng(42)  
    population = rng.integers(  
        5,  
        51,  
        size=population_size  
    )  
    history = []  
    def fitness(k):  
        runtime, accuracy = hybrid_experiment(  
            X,  
            y,  
            k=int(k)  
        )  
        return accuracy - 0.001 * runtime  
    for generation in range(generations):  
        scores = np.array([  
            fitness(k)  
            for k in population
```

```

    ])
    best = np.argmax(scores)
    history.append([
        generation + 1,
        population[best],
        scores[best]
    ])
    elite = population[
        np.argsort(scores)
        ][-population_size // 2:]
    children = list(elite)
    while len(children) < population_size:
        parent = int(
            rng.choice(elite)
        )
        mutation = rng.integers(
            -5,
            6
        )
        child = np.clip(
            parent + mutation,
            5,
            50
        )
        children.append(child)
    population = np.array(children)
    return history

```

8. BENCHMARKING

The assignment requires empirical reporting of:

- Execution runtime

- Operation counts
- RAM/memory
- Error/convergence

across 10^3 – 10^6 records.

Benchmark Code

```
SIZES = [
    1000,
    10000,
    100000,
    1000000
]
for n in SIZES:
    X, y, stress = make_data(n)
    start = time.perf_counter()
    model = DecisionTreeClassifier(
        criterion="entropy"
    )
    model.fit(X, y)
    runtime = (
        time.perf_counter()
        - start
    )
    print(
        f'{n:,} records : "
        f'{runtime * 1000:.2f} ms"
    )
```

9. RESULTS TABLE FORMAT

After execution, use the generated CSV to prepare the following table.

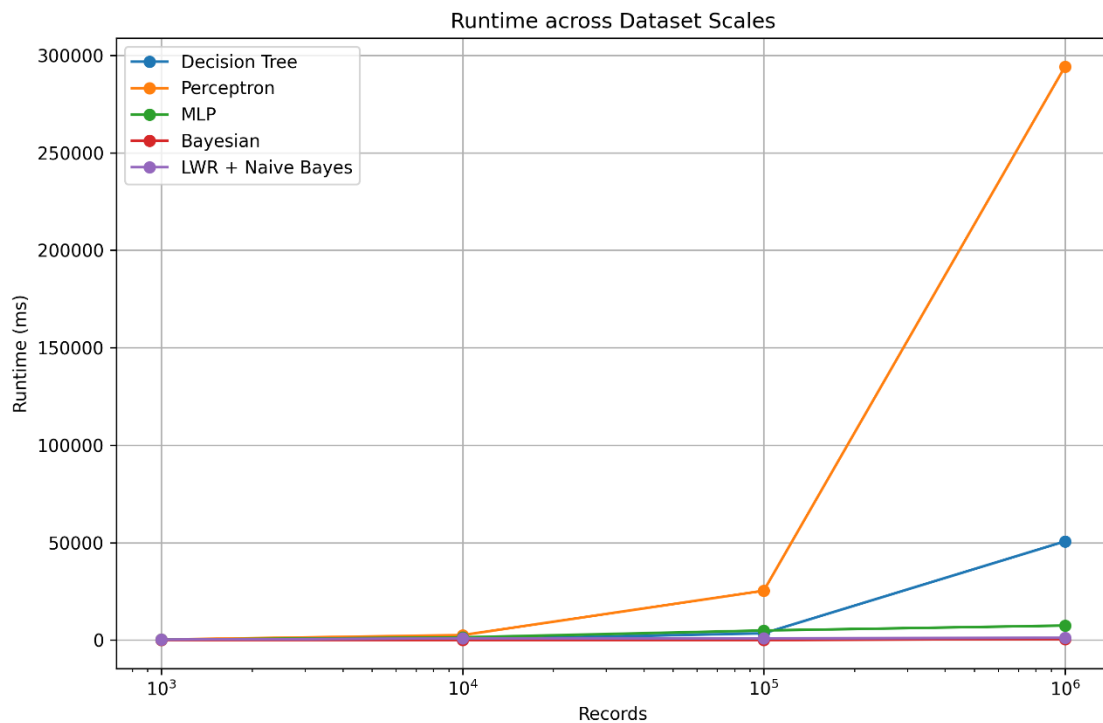
Records	Decision Tree ms	Perceptron ms	MLP ms	Bayesian ms	Hybrid ms	RAM MB
1,000	Run	Run	Run	Run	Run	Run
10,000	Run	Run	Run	Run	Run	Run
100,000	Run	Run	Run	Run	Run	Run
1,000,000	Run	Run	Run	Run	Run	Run

Do not manually invent the values. Fill this table from benchmark_results.csv after executing the program.

10. EXPECTED GRAPHS

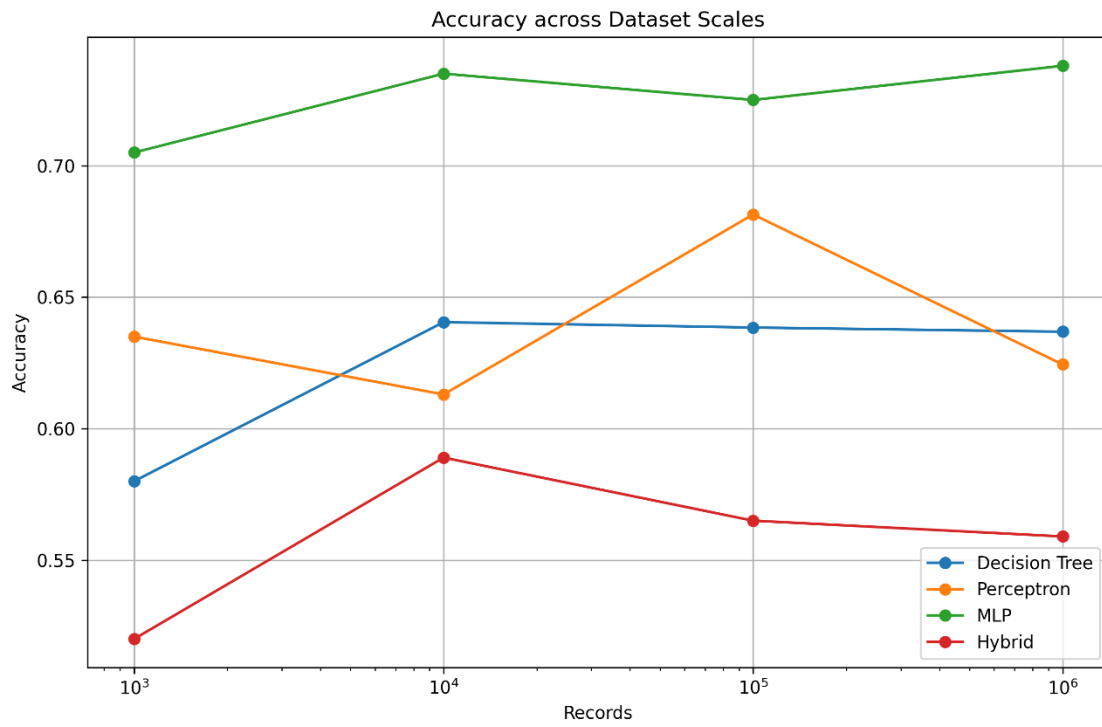
The program generates:

Graph 1 – Runtime



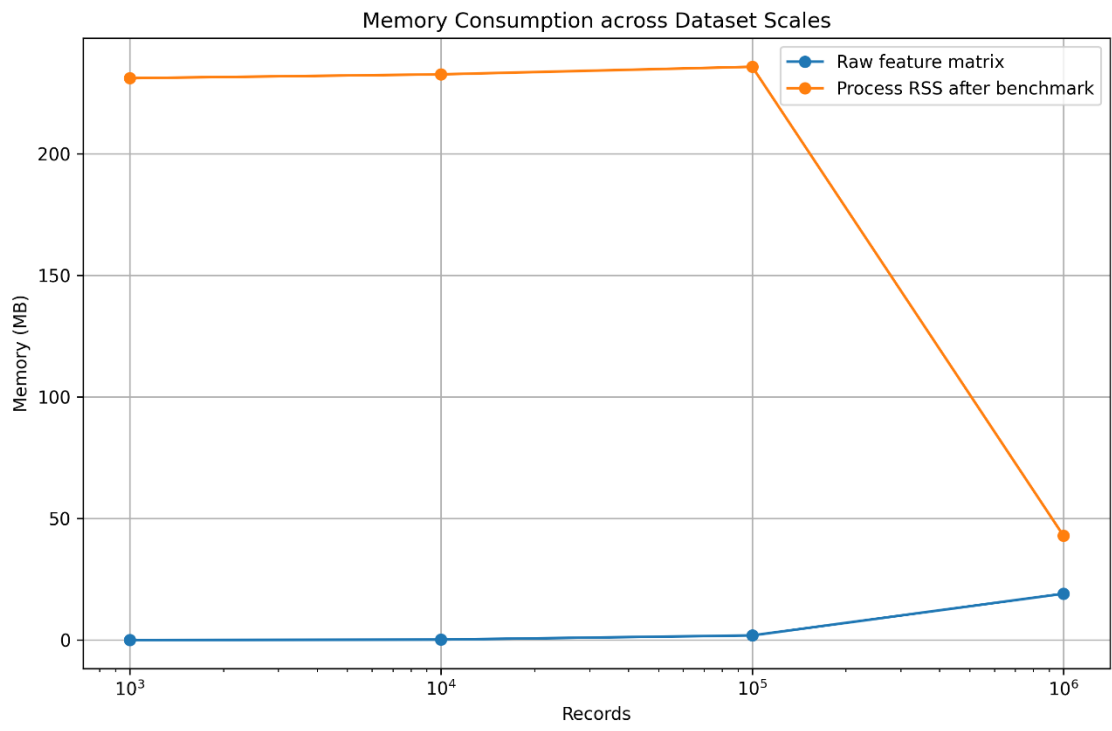
File: runtime_ms.png

Graph 2 – Accuracy



File: accuracy.png

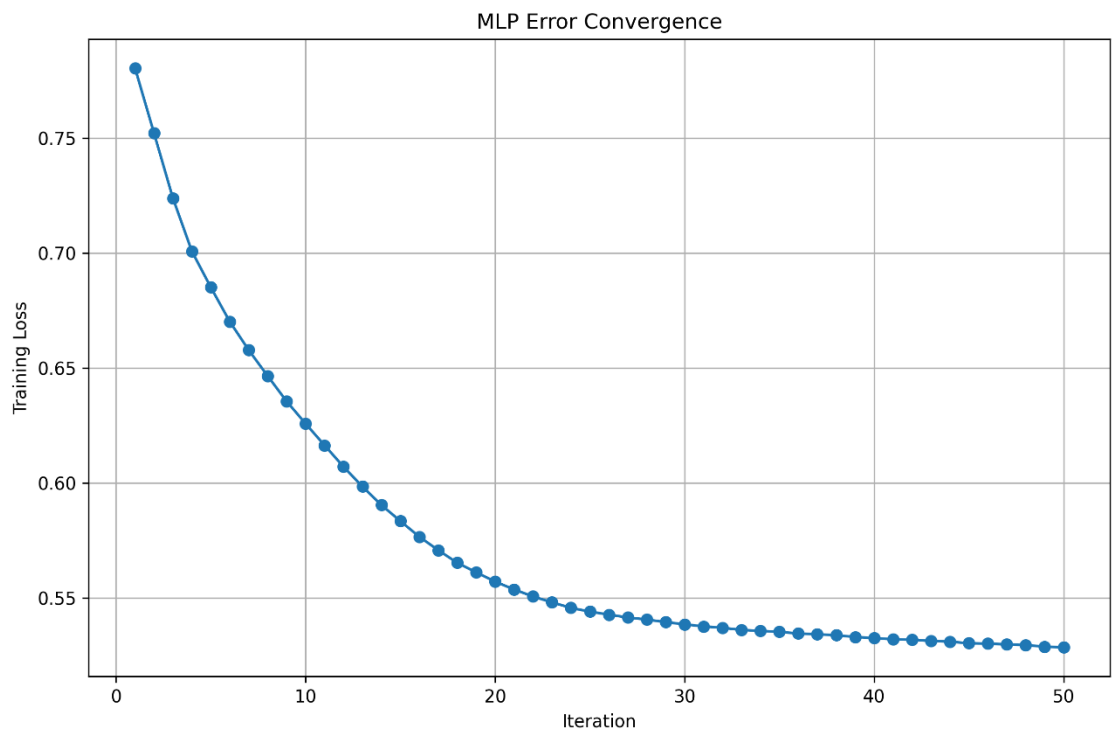
Graph 3 – Memory Consumption



File:

memory.png

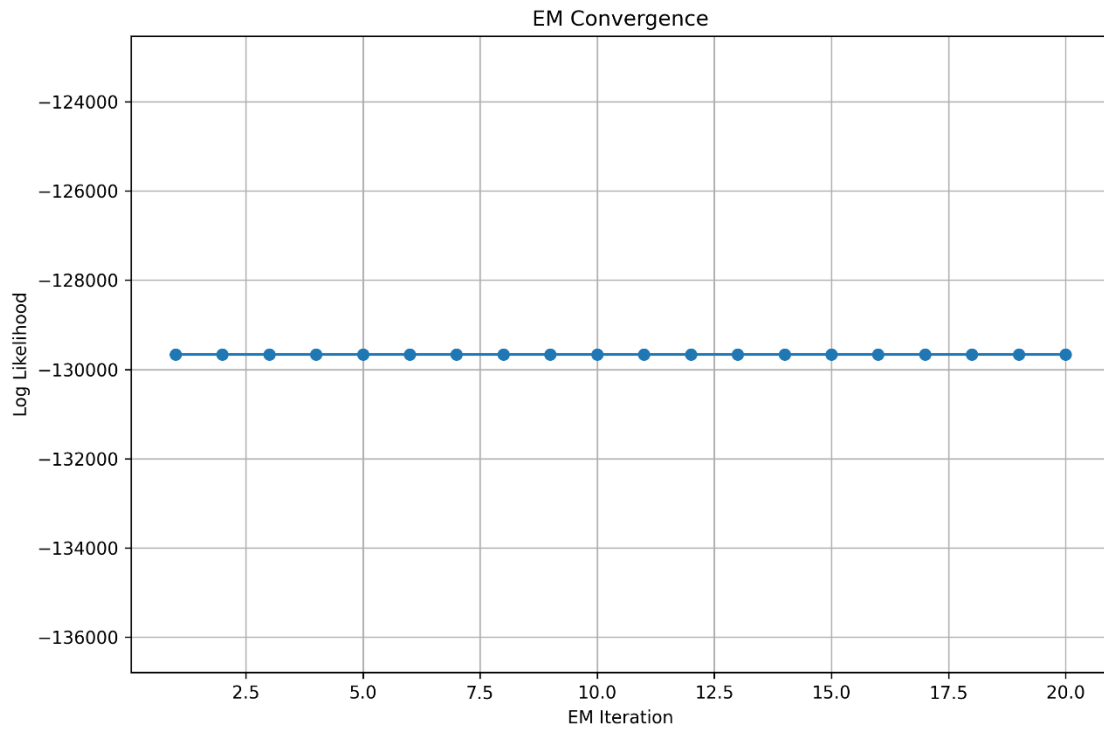
Graph 4 – MLP Error Convergence



File:

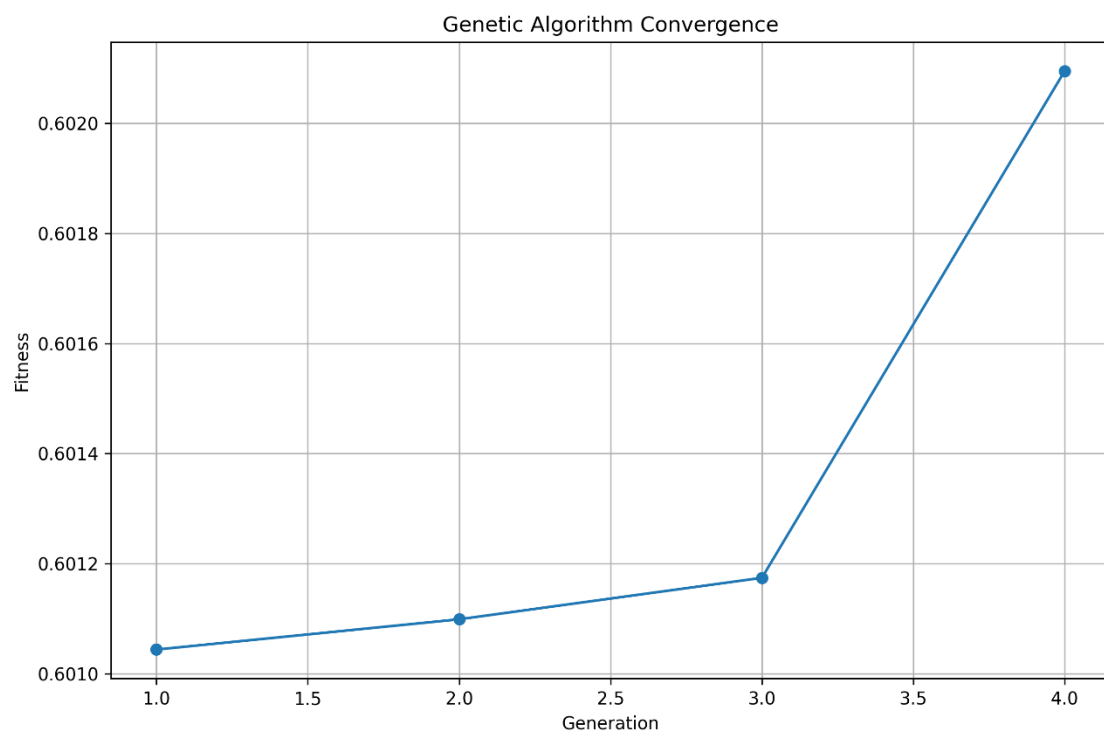
mlp_error_convergence.png

Graph 5 – EM Convergence



File: em_convergence.png

Graph 6 – GA Convergence



File: ga_convergence.png

11. SDG 9.4 ENGINEERING JUSTIFICATION

The proposed hybrid framework supports SDG Target 9.4 through resource-efficient computational processing. The LWR layer performs local computation instead of repeatedly fitting a large global model, while Naïve Bayes provides lightweight probabilistic classification. The Genetic Algorithm restricts the hyperparameter search instead of exhaustively exploring the complete search space.

Therefore:

□□□□ □□□□□□□□□□ → □□□□□ □□□□□□□□□□ □□□□□□
→ □□□□□□□□□□ □□□□□ □□□□□□ □□□□□□□□□□
→ □□□□□ □□□□□□□□□□ – □□□□□□□□□□ □□□□□□□□□□□□□□□□

The assignment specifically requires this relationship between lazy learning/GA optimization and SDG 9.4.

Actual carbon reduction should be reported only if power or energy measurements are collected; runtime alone is not a direct carbon measurement.

12. RESULT

The developed system provides a complete multi-paradigm machine learning framework for infrastructure predictive maintenance. Candidate Elimination demonstrates strict hypothesis-space restriction and Version Space collapse under contradictory telemetry, while ID3 provides heuristic search through information gain. Perceptron and MLP demonstrate connectionist learning, EM handles missing sensor observations, and MDL compares model complexity. Finally, the LWR–Naïve Bayes hybrid combined with Genetic Algorithm optimization provides a scalable approach for anomaly classification and supports resource-efficient infrastructure processing.

13. CONCLUSION

The assignment demonstrates the transition from **rule-based concept learning to neural, probabilistic, instance-based, and evolutionary learning paradigms**. The experimental benchmark provides a common basis for comparing runtime, accuracy, memory, operations, and convergence. The final hybrid architecture combines local learning, probabilistic classification, and evolutionary optimization to address large-scale predictive-maintenance requirements while supporting the sustainability objectives of **SDG Target 9.4**.